ML Numerical by MPS

🔶 Problem Statement:

Suppose you have a dataset of 14 examples (like the classic "Play Tennis" dataset), with the following class distribution:

- 9 Yes
- 5 No

Now, you are considering splitting on the attribute Outlook, which has 3 values:

Outlook	Yes	No
Sunny	2	3
Overcast	4	0
Rain	3	2

Let's calculate the Information Gain for splitting on `Outlook`.

✅ Step 1: Calculate Entropy of the Parent (Before Split)

$$\text{Entropy}(S) = -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}})$$

$$= -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right)$$

$$\approx -0.643 \cdot \log_2(0.643) - 0.357 \cdot \log_2(0.357)$$

$$\approx -0.643 \cdot (-0.643) - 0.357 \cdot (-1.485)$$

$$= 0.409 + 0.530 = \boxed{0.939}$$

✅ Step 2: Calculate Entropy for Each Subset

a) Sunny (2 Yes, 3 No)

$$\text{Entropy} = -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right)$$

$$= -0.4 \cdot (-1.322) - 0.6 \cdot (-0.737) = 0.528 + 0.442 = \boxed{0.970}$$

b) Overcast (4 Yes, 0 No)

All one class ⇒ Entropy = \boxed{0.0}

c) Rain (3 Yes, 2 No)

$$\text{Entropy} = -\frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{2}{5}\log_2\left(\frac{2}{5}\right)$$

$$= -0.6 \cdot (-0.737) - 0.4 \cdot (-1.322) = 0.442 + 0.528 = \boxed{0.970}$$

✅ Step 3: Calculate Weighted Entropy After Split

$$\text{Info}_{\text{Outlook}} = \frac{5}{14} \cdot 0.970 + \frac{4}{14} \cdot 0.0 + \frac{5}{14} \cdot 0.970$$

$$= \frac{10}{14} \cdot 0.970 = 0.714 \cdot 0.970 \approx \boxed{0.693}$$

✅ Step 4: Calculate Information Gain

$$\text{Information Gain} = \text{Entropy(Parent)} - \text{Entropy(After Split)}$$

$$= 0.939 - 0.693 = \boxed{0.246}$$

🟩 Final Answer:

Information Gain for Outlook = 0.246

ML Numerical by MPS

🔶 Problem:

You have a training dataset of emails. Here's the simplified data:

Word	Count in Spam	Count in Ham
Offer	4	1
Win	3	1
Hello	0	3

- Total words in spam emails = 20
- Total words in ham (not spam) emails = 30
- Vocabulary size = 6 (Offer, Win, Hello, and 3 other unique words)

Now, you get a new email:
"Win an Offer"

We want to calculate:

- $P(\text{Spam}|\text{Win, Offer})$
- $P(\text{Ham}|\text{Win, Offer})$

And predict whether it's spam.

✅ Step 1: Prior Probabilities

Assume:

- $P(\text{Spam}) = 0.5,$
- $P(\text{Ham}) = 0.5$

✅ Step 2: Likelihood with Laplace Smoothing

For Spam**:**

$$P(\text{Win}|\text{Spam}) = \frac{3+1}{20+6} = \frac{4}{26}$$

$$P(\text{Offer}|\text{Spam}) = \frac{4+1}{20+6} = \frac{5}{26}$$

For Ham:

$$P(\text{Win}|\text{Ham}) = \frac{1+1}{30+6} = \frac{2}{36}$$

$$P(\text{Offer}|\text{Ham}) = \frac{1+1}{30+6} = \frac{2}{36}$$

✅ Step 3: Naive Bayes Score (Ignore P(X), Compare Numerators)

Spam Score:

$$P(\text{Spam}) \cdot P(\text{Win}|\text{Spam}) \cdot P(\text{Offer}|\text{Spam}) = 0.5 \cdot \frac{4}{26} \cdot \frac{5}{26} = 0.5 \cdot \frac{20}{676} = \frac{10}{676}$$

Ham Score:

$$0.5 \cdot \frac{2}{36} \cdot \frac{2}{36} = 0.5 \cdot \frac{4}{1296} = \frac{2}{1296}$$

✅ Step 4: Compare and Predict

Convert to decimals:

- Spam score ≈ 0.0148
- Ham score ≈ 0.0015

🔔 Prediction: Spam (since spam score is higher)

🔶 Problem:

You have a simple neural network with:

- 1 input neuron
- 1 hidden neuron (with sigmoid activation)
- 1 output neuron (also sigmoid)

Given:

- Input: $x = 1$
- True output: $y = 0$
- Initial weights:
  - $w_1 = 0.5$ (input → hidden)
  - $w_2 = 0.8$ (hidden → output)

- Learning rate: $\eta = 0.1$

We will do 1 step of forward + backpropagation and update the weights.

✅ Step 1: Forward Pass

a. Hidden Layer Activation

$$z_1 = x \cdot w_1 = 1 \cdot 0.5 = 0.5$$

$$a_1 = \sigma(z_1) = \frac{1}{1 + e^{-0.5}} \approx 0.622$$

b. Output Layer Activation

$$z_2 = a_1 \cdot w_2 = 0.622 \cdot 0.8 = 0.498$$

$$\hat{y} = \sigma(z_2) = \frac{1}{1 + e^{-0.498}} \approx 0.622$$

✅ Step 2: Compute Loss (MSE)

$$E = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(0 - 0.622)^2 = 0.193$$

✅ Step 3: Backpropagation

a. Output Layer Error

$$\delta_2 = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) = (0.622 - 0) \cdot 0.622 \cdot (1 - 0.622)$$

$$= 0.622 \cdot 0.622 \cdot 0.378 \approx 0.146$$

b. Hidden Layer Error

$$\delta_1 = \delta_2 \cdot w_2 \cdot a_1(1 - a_1)$$

$$= 0.146 \cdot 0.8 \cdot 0.622 \cdot (1 - 0.622) \approx 0.146 \cdot 0.8 \cdot 0.235 \approx 0.027$$

✅ Step 4: Weight Updates

$$w_2 := w_2 - \eta \cdot \delta_2 \cdot a_1 = 0.8 - 0.1 \cdot 0.146 \cdot 0.622 \approx 0.8 - 0.009 \approx \boxed{0.791}$$

$$w_1 := w_1 - \eta \cdot \delta_1 \cdot x = 0.5 - 0.1 \cdot 0.027 \cdot 1 = \boxed{0.4973}$$

🟩 Final Answer:

- Updated $w_1 = 0.4973$
- Updated $w_2 = 0.791$

ML Numerical by MPS

🔶 Problem:

A binary classification model was tested on 100 instances. The results are as follows:

	Predicted: Yes	Predicted: No
Actual: Yes	50	10
Actual: No	5	35

Let's calculate:

1. Accuracy
2. Precision
3. Recall (Sensitivity)
4. F1-Score
5. Specificity

✅ Step 1: Identify TP, TN, FP, FN

- TP (True Positive) = 50
- FN (False Negative) = 10
- FP (False Positive) = 5
- TN (True Negative) = 35

✅ Step 2: Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{50 + 35}{100} = \boxed{85\%}$$

✅ Step 3: Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{50}{50 + 5} = \frac{50}{55} \approx \boxed{0.909}$$

✅ Step 4: Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{50}{50 + 10} = \frac{50}{60} = \boxed{0.833}$$

✅ Step 5: F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{0.909 \cdot 0.833}{0.909 + 0.833} \approx 2 \cdot \frac{0.757}{1.742} \approx \boxed{0.869}$$

✅ Step 6: Specificity (True Negative Rate)

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{35}{35 + 5} = \frac{35}{40} = \boxed{0.875}$$

🟩 Final Summary:

Metric	Value
Accuracy	0.85
Precision	0.909
Recall	0.833
F1-Score	0.869
Specificity	0.875

🔶 Problem:

You are given a dataset with 4 training examples:

Sample	x	y (label)
1	1	+1
2	2	+1
3	3	–1

Sample	x	y (label)
4	4	–1

Assume:

- You will train a weak classifier $h_t(x)$ that predicts:
  - +1 if x < 3, else –1
- Initial weights:

$$w_1 = w_2 = w_3 = w_4 = \frac{1}{4}$$

Let's calculate for iteration 1:

1. Weighted error ε_t
2. Classifier weight α_t
3. Update all sample weights

✅ Step 1: Predictions from weak classifier

Classifier: $h(x) = +1$ if $x < 3$, else –1
Apply to all:

Sample	x	y	h(x)	Correct?
1	1	+1	+1	☑
2	2	+1	+1	☑
3	3	–1	–1	☑
4	4	–1	–1	☑

All predictions are correct, so:

$$\varepsilon_t = \sum w_i \cdot I(h(x_i) \neq y_i) = 0$$

But to avoid infinite alpha, we add a tiny epsilon (e.g., 0.0001):

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right) = \frac{1}{2} \log\left(\frac{1 - 0.0001}{0.0001}\right) = \frac{1}{2} \log(9999) \approx \boxed{4.6}$$

✅ Step 2: Update Weights

New weights:

$$w_i := w_i \cdot e^{-\alpha_t \cdot y_i \cdot h(x_i)}$$

Since all predictions are correct $y_i = h(x_i)$, so:

$$w_i^{\text{new}} = w_i \cdot e^{-\alpha_t \cdot 1} = \frac{1}{4} \cdot e^{-4.6} \approx \frac{1}{4} \cdot 0.01 \approx 0.0025$$

Now normalize weights:

$$Z = \sum w_i^{\text{new}} = 4 \cdot 0.0025 = 0.01 \Rightarrow \text{Final weight for each} = \frac{0.0025}{0.01} = \boxed{0.25}$$

→ All weights stay the same in this iteration.

🟩 Final Summary (Iteration 1):

- Error ε = 0
- Alpha α = 4.6
- New Weights = [0.25, 0.25, 0.25, 0.25]

This is an ideal case — in next rounds, errors will occur and weights shift toward misclassified points.

ML Numerical by MPS

🔶 Problem:

You are given the following data points:

x	y
1	2
2	3
3	5
4	4
5	6

Find the regression line equation:

$$y = mx + c$$

✅ Step 1: Calculate Required Sums

$$n = 5, \quad \sum x = 1 + 2 + 3 + 4 + 5 = 15, \quad \sum y = 2 + 3 + 5 + 4 + 6 = 20$$

$$\sum x^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$$

$$\sum xy = 1 \cdot 2 + 2 \cdot 3 + 3 \cdot 5 + 4 \cdot 4 + 5 \cdot 6 = 2 + 6 + 15 + 16 + 30 = 69$$

✅ Step 2: Calculate Slope (m)

$$m = \frac{n\sum xy - \sum x \sum y}{n\sum x^2 - (\sum x)^2} = \frac{5 \cdot 69 - 15 \cdot 20}{5 \cdot 55 - 15^2} = \frac{345 - 300}{275 - 225} = \frac{45}{50} = \boxed{0.9}$$

✅ Step 3: Calculate Intercept (c)

$$c = \frac{\sum y - m\sum x}{n} = \frac{20 - 0.9 \cdot 15}{5} = \frac{20 - 13.5}{5} = \frac{6.5}{5} = \boxed{1.3}$$

🟩 Final Regression Line:

$$\boxed{y = 0.9x + 1.3}$$

ML Numerical by MPS

🔶 Problem:

You're given a separating hyperplane:

$$w \cdot x + b = 0 \quad \text{where} \quad w = [2, -3],\ b = 6$$

And a test point:

$$x_0 = [4, 2]$$

Find the perpendicular distance from this point to the hyperplane.

✅ Step 1: Use the Distance Formula

$$\text{Distance} = \frac{|w \cdot x_0 + b|}{||w||}$$

✅ Step 2: Calculate Dot Product $w \cdot x_0$

$$w \cdot x_0 = 2 \cdot 4 + (-3) \cdot 2 = 8 - 6 = 2$$

$$\Rightarrow w \cdot x_0 + b = 2 + 6 = 8$$

✅ Step 3: Calculate $||w||$ (L2 Norm)

$$||w|| = \sqrt{2^2 + (-3)^2} = \sqrt{4 + 9} = \sqrt{13}$$

✅ Step 4: Plug into Formula

$$\text{Distance} = \frac{|8|}{\sqrt{13}} \approx \frac{8}{3.606} \approx \boxed{2.218}$$

🟩 Final Answer:

The distance from the point $x_0 = [4, 2]$ to the hyperplane is approximately 2.218 units.

ML Numerical by MPS

🔶 Problem:

You are given:

- Input $x = 2$
- Weight $w = 0.8$
- Bias $b = -1.2$
- True label $y = 1$

Use:

- Sigmoid function for prediction
- Binary cross-entropy loss
- Learning rate $\eta = 0.1$

👉 Perform 1 forward pass and 1 weight update using gradient descent.

✅ Step 1: Compute z (Linear Combination)

$$z = w \cdot x + b = 0.8 \cdot 2 - 1.2 = 1.6 - 1.2 = 0.4$$

✅ Step 2: Compute $\hat{y}$ using Sigmoid

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-0.4}} \approx 0.5987$$

✅ Step 3: Compute Binary Cross-Entropy Loss

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) = -1 \cdot \log(0.5987) = \boxed{0.513}$$

✅ Step 4: Compute Gradients

The gradient w.r.t. weight:

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x = (0.5987 - 1) \cdot 2 = -0.4013 \cdot 2 = \boxed{-0.8026}$$

The gradient w.r.t. bias:

$$\frac{\partial L}{\partial b} = \hat{y} - y = -0.4013$$

✅ Step 5: Update Parameters

$$w_{\text{new}} = w - \eta \cdot \frac{\partial L}{\partial w} = 0.8 - 0.1 \cdot (-0.8026) = 0.8 + 0.08026 = \boxed{0.8803}$$

$$b_{\text{new}} = b - \eta \cdot \frac{\partial L}{\partial b} = -1.2 + 0.04013 = \boxed{-1.1599}$$

🟩 Final Answer:

- Updated weight: 0.8803
- Updated bias: –1.1599
- Loss: 0.513

ML Numerical by MPS

🔶 Problem:

You're given a small dataset:

ID	Age	Income ( )	Gender	Purchased
1	25	50000	Male	No
2	30	60000	Female	Yes
3	35	NaN	Female	Yes
4	40	80000	Male	No

Apply the following preprocessing steps:

1. Handle missing values in "Income"
2. Apply Min-Max Scaling on "Age"
3. Use Label Encoding for "Gender"
4. Use One-Hot Encoding for "Purchased"

✅ Step 1: Handle Missing Values (Imputation)

Column "Income" has a missing value at ID 3.
We'll use mean imputation:

Known values: 50000, 60000, 80000
Mean = $\frac{50000+60000+80000}{3} = \frac{190000}{3} = 63333.33$

👉 Replace NaN with ₹63,333.33

✅ Step 2: Min-Max Scaling for "Age"

Min = 25, Max = 40
Min-Max Formula:

$$x' = \frac{x - \min}{\max - \min}$$

ID	Age	Scaled Age
1	25	$\frac{25-25}{15} = 0$

ID	Age	Scaled Age
2	30	$\frac{30-25}{15} = 0.33$
3	35	$\frac{35-25}{15} = 0.67$
4	40	$\frac{40-25}{15} = 1$

✅ Step 3: Label Encoding for "Gender"

Assign:

- Male → 0
- Female → 1

Gender	Encoded
Male	0
Female	1

✅ Step 4: One-Hot Encoding for "Purchased"

Column has values: Yes, No

Create 1 column: Purchased_Yes

Purchased	Purchased_Yes
No	0
Yes	1

🟩 Final Transformed Table:

ID	Scaled Age	Income	Gender (Label)	Purchased_Yes
1	0.00	50000	0	0
2	0.33	60000	1	1
3	0.67	63333.33	1	1

ID	Scaled Age	Income	Gender (Label)	Purchased_Yes
4	1.00	80000	0	0

ML Numerical by MPS

🔶 1. Convolution Operation

- Applies filters (kernels) over input to extract features.
- Each filter slides over input to compute dot product.

Formula (Output Size):

$$\text{Output} = \left\lfloor \frac{(W - K + 2P)}{S} \right\rfloor + 1$$

Where:

- W: Input size
- K: Kernel size
- P: Padding
- S: Stride

🔶 2. Stride (S)

- Defines step size of filter movement.
- Stride = 1 → normal, overlaps
- Stride > 1 → skips positions → smaller output

Example:
Input = 7×7, Kernel = 3×3, Stride = 2 → Output = 3×3

🔶 3. Padding (P)

- Adds border of zeros around input to:
  - Preserve spatial size ("same" padding)
  - Avoid size reduction

Types:

- Valid Padding: No padding → Output shrinks
- Same Padding: Adds padding to keep output same size as input

---

🔶 4. Pooling Layers

- Reduces dimensions, helps generalization
- Types:
  - Max Pooling: Takes max value
  - Average Pooling: Takes average

Example:
2×2 Max Pooling over:

$$\begin{bmatrix} 1 & 3 \\ 4 & 2 \end{bmatrix} \Rightarrow 4$$

🔶 5. Depth (Channels)

- Each filter produces 1 feature map
- More filters = More depth in output

🔶 6. ReLU Activation

- Applied after convolution:

$$\text{ReLU}(x) = \max(0, x)$$

- Adds non-linearity

🔶 7. Flattening & Fully Connected Layers

- After convolution + pooling, data is flattened and passed to dense layers for classification.

🔶 Summary Table:

Concept	What it Does	Key Point
Stride	Moves filter over input	Larger stride = smaller output
Padding	Adds zeros to borders	Keeps size same (if needed)
Pooling	Downsamples data	Reduces overfitting
ReLU	Non-linear activation	Speeds up training

ML Numerical by MPS

🔶 Problem:

You are given:

- Input image: 32 × 32
- Filter (kernel) size: 5 × 5
- Stride: 1
- Padding: 2

Find the output size after applying 1 convolutional layer with these parameters.

✅ Step 1: Use the Output Size Formula

$$\text{Output Size} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

Where:

- $W = 32$ (input size)
- $K = 5$ (kernel size)
- $P = 2$ (padding)
- $S = 1$ (stride)

✅ Step 2: Plug in Values

$$\text{Output Size} = \left\lfloor \frac{32 - 5 + 2 \cdot 2}{1} \right\rfloor + 1 = \left\lfloor \frac{31}{1} \right\rfloor + 1 = 31 + 1 = \boxed{32}$$

🟩 Final Answer:

Output feature map = 32 × 32

✅ That means "same padding" preserved the spatial size.

---

1. Max Pooling Example
2. CNN with Multiple Filters Example

🔶 Part 1: Max Pooling

You are given a 4×4 feature map:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 9 & 8 & 4 & 3 \\ 7 & 2 & 5 & 6 \end{bmatrix}$$

Apply 2×2 Max Pooling, stride = 2

✅ Step 1: Divide the input into 2×2 non-overlapping regions:

Region 1	Region 2
1 3 5 6	2 4 1 2
9 8 7 2	4 3 5 6

✅ Step 2: Take max of each region:

$$\begin{bmatrix} \max(1,3,5,6) = 6 & \max(2,4,1,2) = 4 \\ \max(9,8,7,2) = 9 & \max(4,3,5,6) = 6 \end{bmatrix}$$

🟩 Output After Max Pooling:

$$\boxed{\begin{bmatrix} 6 & 4 \\ 9 & 6 \end{bmatrix}}$$

🔶 Part 2: CNN with Multiple Filters

Suppose:

- Input image = 28×28×1 (grayscale)
- You use 8 filters of size 3×3, stride = 1, padding = 1

✅ Output Size:

Use same formula for 1 filter:

$$\text{Output size} = \left\lfloor \frac{28 - 3 + 2 \cdot 1}{1} \right\rfloor + 1 = 28$$

So each filter produces 28×28

✅ Total Output:

Since you have 8 filters, output will be:

$$\boxed{28 \times 28 \times 8}$$
